

Lab 5 — Single-Layer Perceptron (AND / OR Gate)

Aim

To implement a single-layer perceptron from scratch to learn the AND and OR logic gates.

Algorithm / Procedure

1. Initialize the weights and bias to zero.
2. Set the learning rate and number of epochs.
3. For each training sample:
 - Add a bias input.
 - Calculate the weighted sum.
 - Apply the threshold activation function.
4. Calculate the error between the target and predicted output.
5. Update the weights using:

$$\Delta w = \eta(t - o)x$$

6. Repeat the process for the specified number of epochs.
7. Test the trained perceptron with all input combinations.
8. Display the learned weights and predicted outputs for AND and OR gates.

Program

```
import numpy as np

# Perceptron Training Function

def perceptron_train(X, y, epochs=20, lr=0.1):

    # +1 for bias

    w = np.zeros(X.shape[1] + 1)

    for _ in range(epochs):

        for xi, target in zip(X, y):
```

```

        # Add bias

        xi_b = np.insert(xi, 0, 1)

        # Calculate output

        out = 1 if np.dot(w, xi_b) > 0 else 0

        # Update weights

        w += lr * (target - out) * xi_b

    return w

# Input combinations
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

# AND Gate
y_and = np.array([0, 0, 0, 1])

w_and = perceptron_train(X, y_and)

print("Learned weights (AND):", w_and)

print("AND Gate:")

for xi in X:

    xi_b = np.insert(xi, 0, 1)

    output = 1 if np.dot(w_and, xi_b) > 0 else 0

    print(xi, "->", output)

# OR Gate

```

```

y_or = np.array([0, 1, 1, 1])

w_or = perceptron_train(X, y_or)

print("\nLearned weights (OR):", w_or)

print("OR Gate:")

for xi in X:

    xi_b = np.insert(xi, 0, 1)

    output = 1 if np.dot(w_or, xi_b) > 0 else 0

    print(xi, "->", output)

```

OUTPUT

```

... Learned weights (AND): [-0.2  0.2  0.1]
AND Gate:
[0 0] -> 0
[0 1] -> 0
[1 0] -> 0
[1 1] -> 1

Learned weights (OR): [0.  0.1 0.1]
OR Gate:
[0 0] -> 0
[0 1] -> 1
[1 0] -> 1
[1 1] -> 1

```

Result

The single-layer perceptron was successfully implemented from scratch and trained to learn both AND and OR logic gates. The perceptron correctly classified all input combinations.